AiDiy Slack 操作編
操作ガイドとセルフテスト

> 確認範囲：本文には過去の実機・自動試験結果と今回の追補を含みます。今回の実施状況は確認結果の章別台帳を参照してください。今回の全章再実行・全条件合格を示すものではありません。

総合評価

✅ OK（記載した確認範囲）。Windowsの実パネルとSlack専用チャンネルで、起動・自動接続、初回設定、投稿へのAI回答、会話継続、モデル設定の取消と再接続、切断・終了、状態確認等のコマンドを確認しました。会話の継続は同じスレッドで確認しています。

1〜4章が操作ガイド、5章が追加テストです。現行画面19点を掲載しています。Nodeの自動・隔離試験82件、Pythonの14件、DiagのSlack起動7件が成功し、型検査と配布画面の一致も確認しました。PDF版を参照できます。

始める前に

- aidiy_slackとHermesの導入・認証を済ませます。導入とサービス側の設定はSlack READMEを参照します。
- 専用チャンネルと許可利用者を設定します。Botトークンは画面や資料に載せません。
- Diagの起動と作業フォルダ選択はシステム起動編とランチャー編を参照します。

SlackはHTTPSでポーリングし、新しい投稿へスレッドで返信します。通常は3秒間隔です。最後の会話から10分でアイドリングモードに入り、6秒間隔になります。新しい会話を受信すると3秒へ戻ります。Socket ModeやLive音声は使いません。

1. 起動して接続する

1. Diagで作業フォルダを選び、ミニランチャーの「Slack」を押します。端末からはaidiy_slackで起動できます。
2. 「プロジェクト」が選んだフォルダであることを確認します。
3. 初回は自動接続せず、モデル設定画面から始まります。コードはaidiy_hermes固定で、モデルを選んで「接続」を押します。
4. 保存済みの設定がある場合は、起動時に前回の設定で自動接続します。

完了の目印： 左上の「接続済み」と緑の点、右上の「切断」、「CODE ON」、下部の現在のモデル表示です。

ミニランチャーのSlackを押して、選んだプロジェクトのパネルを開きます。

図101：ミニ表示



保存設定がない条件では、未接続の設定画面で停止しました。コードはaidiy_hermes、モデルはautoです。接続前に保存ファイルが作られないことも確認しました。

図102：初回起動



保存済みの設定で起動すると自動接続しました。初回画面の「接続」による保存も確認し、選択内容は元の設定と一致しました。

図103：自動接続



2. 専用チャンネルで話しかける

1. 許可した利用者で、設定した専用テキストチャンネルを開きます。
2. 短い文を普通に投稿します。メンションや接頭辞は不要です。
3. 投稿にAIの回答が返ることと、パネル中央に最後のやり取りが表示されることを確認します。
4. 同じスレッドへ続けて質問し、前の投稿を踏まえた回答かを読みます。

完了の目印： Slack上で回答を読めることです。パネル表示だけで、投稿先への回答成功とは判定しません。パネルには最後のやり取りを1件表示し、約1分で消します。

受信の3秒／6秒は、1回の取得処理が終わってから次の取得までの待ち時間です。投稿の検出とAIの回答生成は別の時間がかかります。新しい投稿には、まず「考えています…」と返信し、回答で書き換えます。

確認にはHermesのautoを使い、「おはよう。合言葉は『銀の雲』です。短く挨拶し、合言葉を確認してください。」と投稿しました。続けて「さっきの合言葉を一言で教えてください。」と尋ねています。

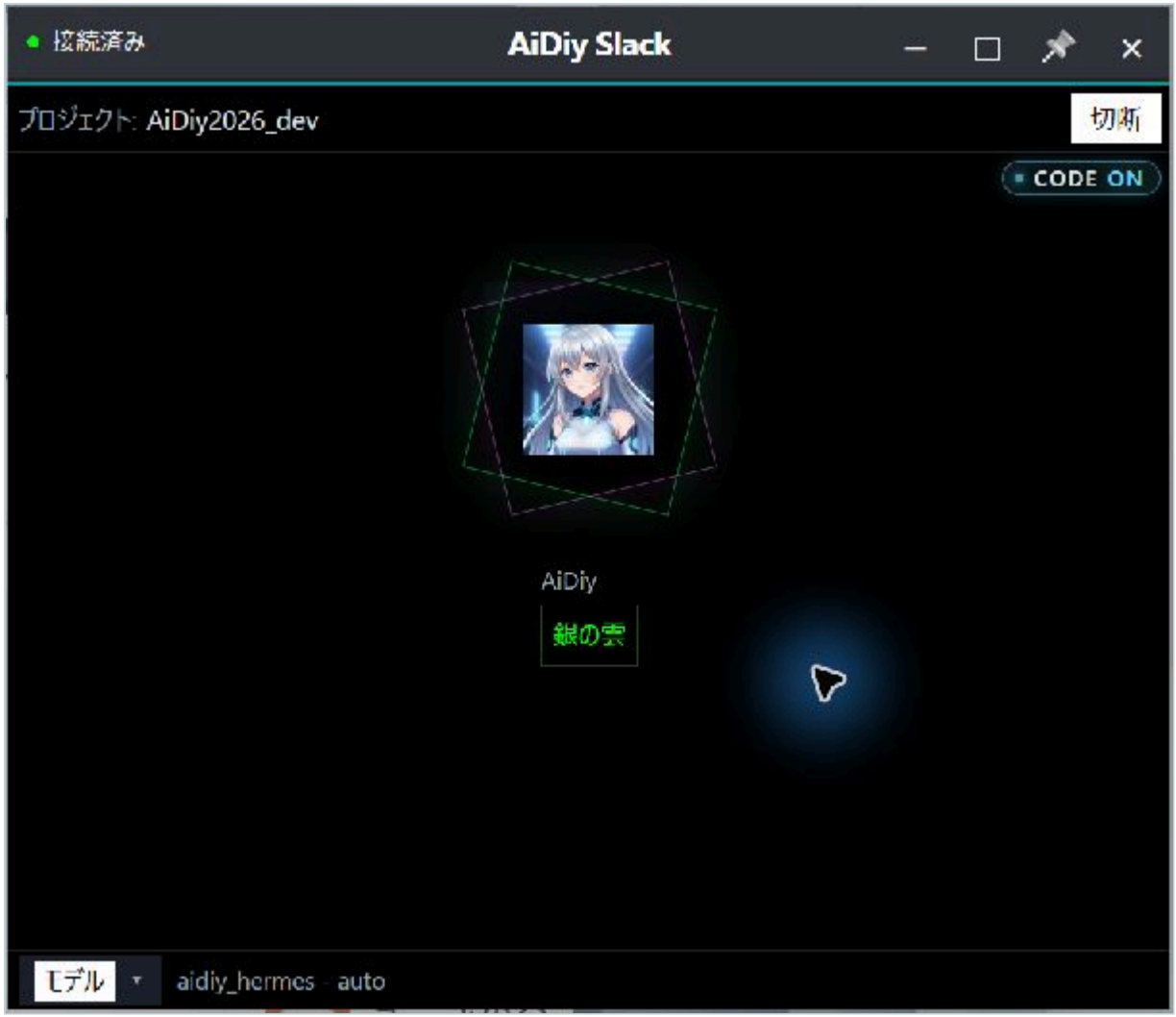
投稿を受け付け、パネルに利用者の本文を表示しました。投稿者は「あなた」と表示します。

図201：投稿表示



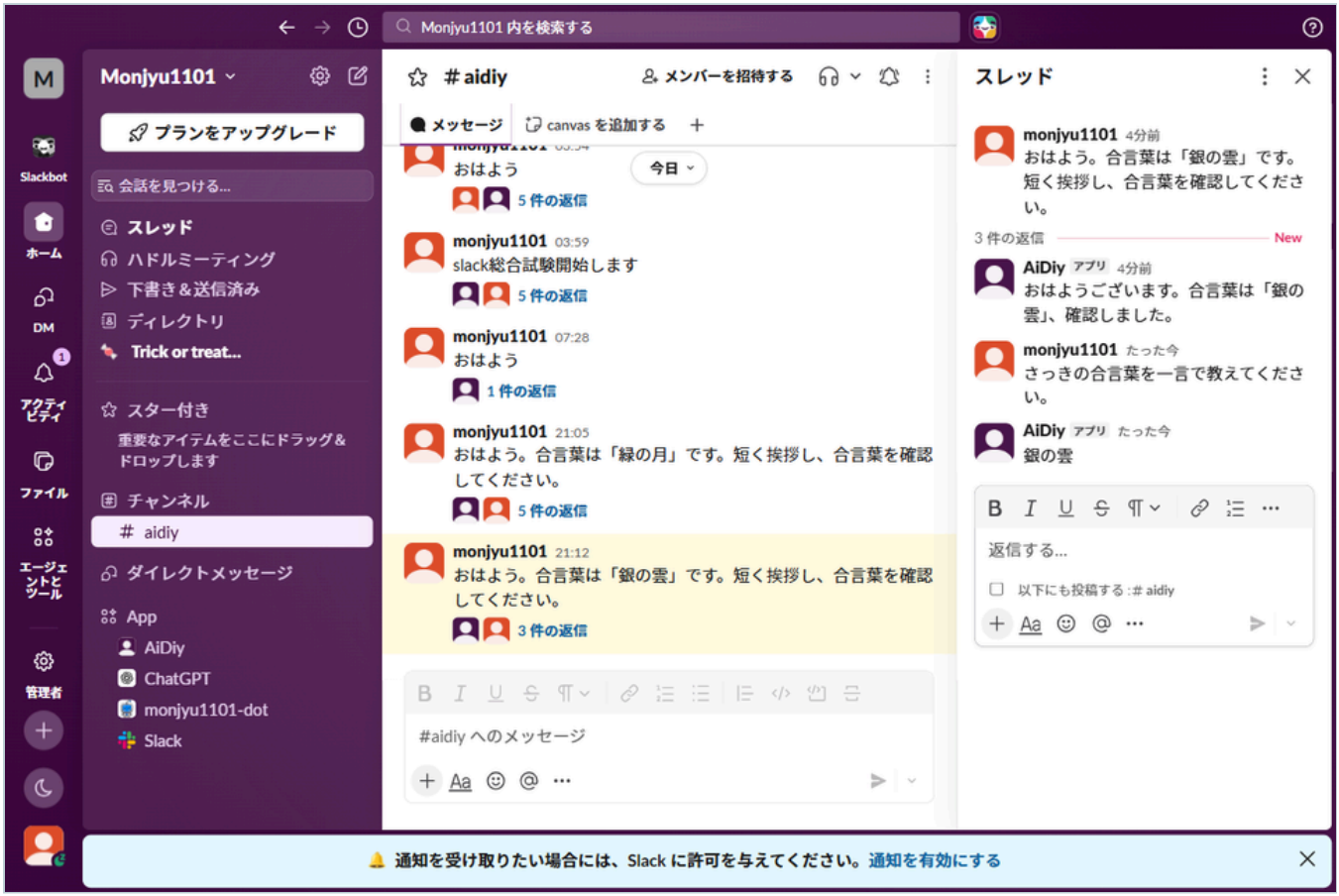
続きの質問に「銀の雲」と返したパネルです。空応答やエラーではなく、入力内容に合う回答を確認しています。

図202：回答表示



Slackの専用チャンネル「aidiy」の同じスレッドで、投稿・AI回答・続きの質問を確認できます。合言葉を質問に書き直さなくても「銀の雲」と返り、会話の文脈が引き継がれました。

図203：スレッド返信



3. モデルを選び直す

1. 下部の「モデル ▾」を押します。接続中は、接続を続けたまま設定欄が開きます。
2. コードは`aidiy_hermes`の固定表示です。モデルを一覧から選びます。
3. 「キャンセル」は変更を戻し、接続を続けます。「適用して再接続」は切断・保存・再接続します。
4. 再び「接続済み」になり、下部のモデル表示が選択内容と一致することを確認します。

設定欄のボタンが見えない場合は、欄内を下へスクロールします。`auto`は自動判定、`same`は変更なし、`codex_cli/same`はCLI指定、`openai_oauth/…`はHermesのプロバイダとモデルの指定です。選択は`~/.aidiy/aidiy_slack.json`に保存します。

接続済みのまま「モデル」を開きました。固定のコード欄と、編集できるモデル欄が表示されます。

図301：接続中のモデル設定



モデルをsameへ変更したところです。「キャンセル」と「適用して再接続」を区別して操作します。

図302：モデル選択例



キャンセルするとautoの表示へ戻り、接続は継続しました。保存ファイルもautoのままでした。

接続済み・autoの表示位置は第2章の図202を参照します。同一画像だけで取消操作の実施を示すものではなく、取消と保存値の確認結果は上記の記述を参照します。

設定欄を開き直し、元のautoで「適用して再接続」を押します。

図303：適用して再接続



再び接続済みとなり、モデルはaidiy_hermes - autoです。保存値の一致も確認しました。

図304：再接続完了



4. 切断・再接続・終了する

1. 右上の「切断」を押します。「未接続」のモデル設定画面に戻ります。
2. 再び使うときは設定を確認し、「接続」を押します。
3. 終了するときは右上の「×」を押します。接続中は終了確認が表示されます。
4. 「キャンセル」なら接続を続け、「終了」ならパネル・Bot・子プロセスを終了します。切断中は確認なしで終了します。

完了の目印： 切断は「未接続」と「接続」、再接続は「接続済み」、終了はパネルが閉じたことを確認します。停止の確認では、このアプリのBotや子プロセスも照合します。

「切断」で未接続の設定画面へ戻りました。

図401：切断



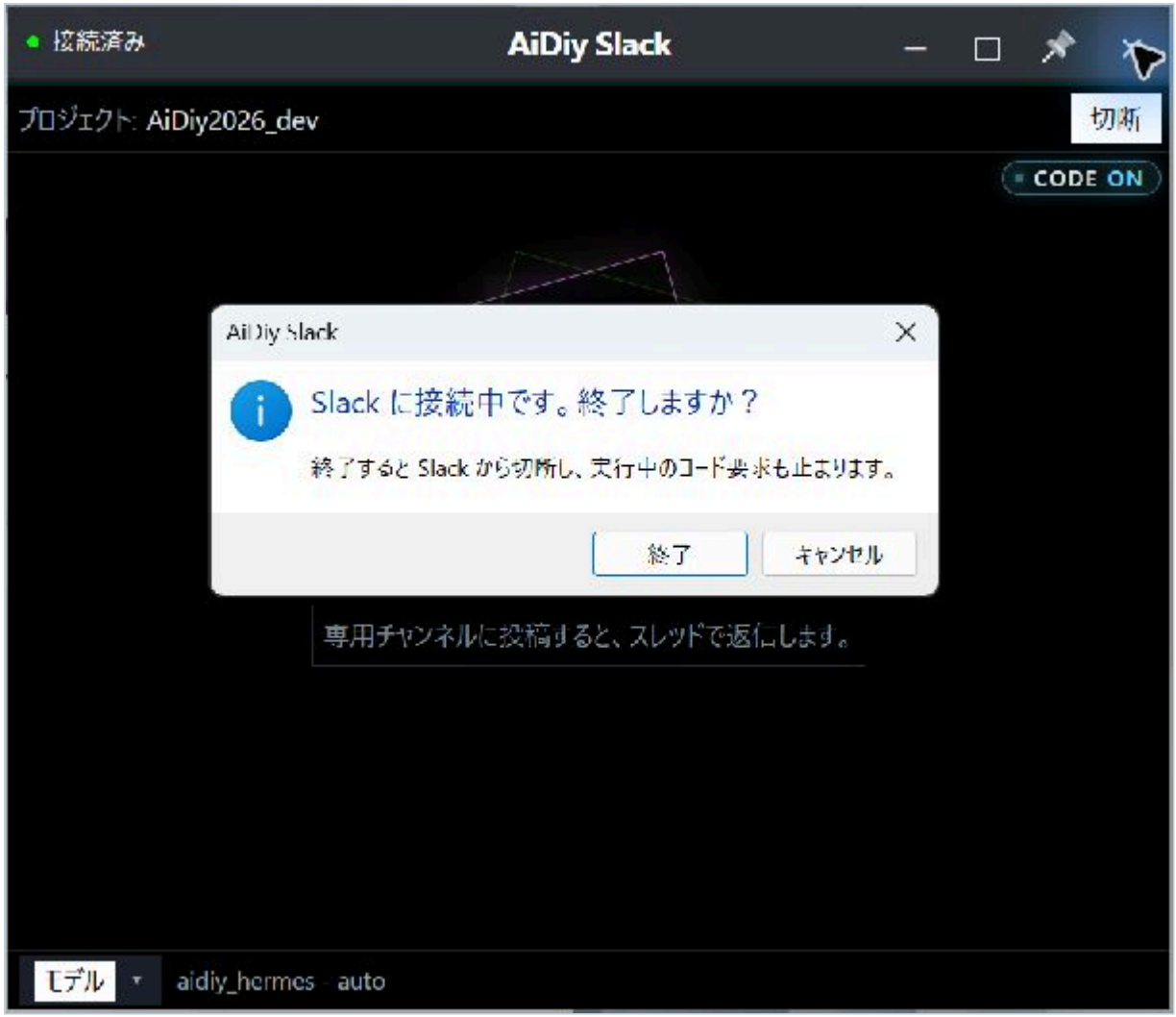
「接続」で再び接続済みとなりました。

図402：再接続



接続中に「×」を押すと、終了確認を表示します。

図403：接続中の終了確認



キャンセルすると確認が閉じ、接続を継続しました。確認を開き直した「終了」ではパネルが閉じました。切断中の「×」は確認なしで終了することも確認済みです。

図404：終了取消で接続継続

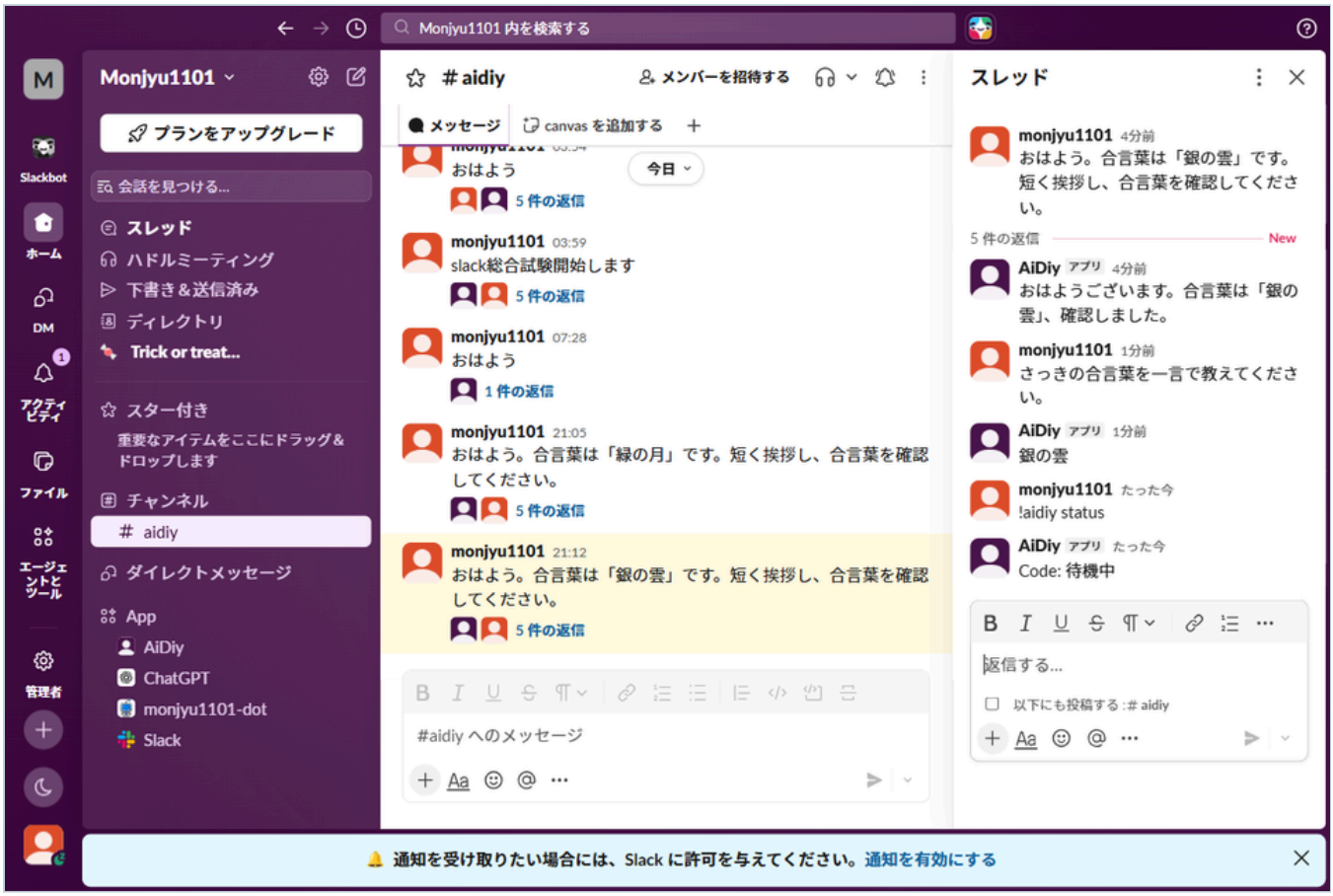


5. 追加テスト

状態確認と会話の操作

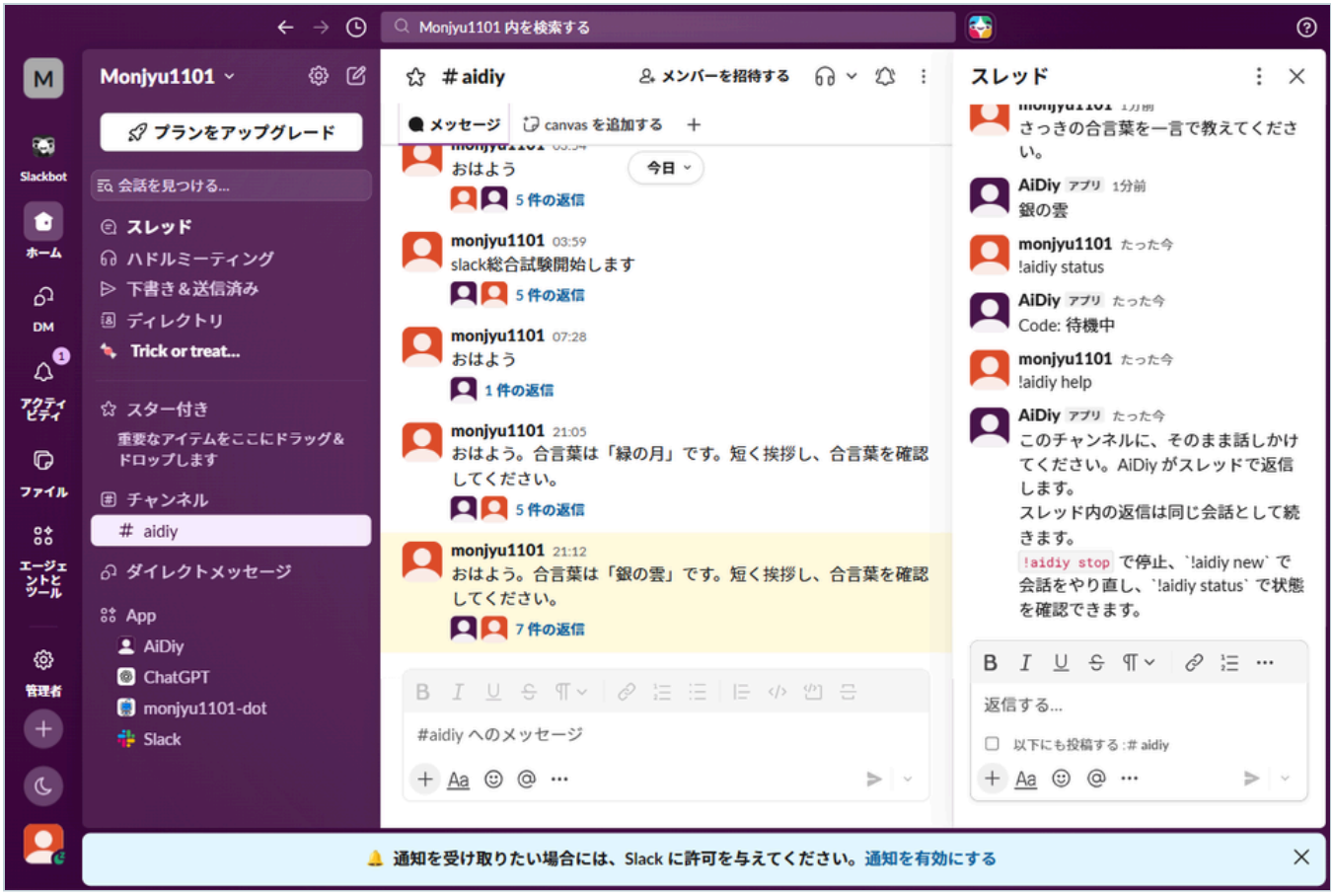
専用チャンネルの同じスレッドへ!aidiy statusを送り、「Code: 待機中」が返りました。

図501：状態コマンド



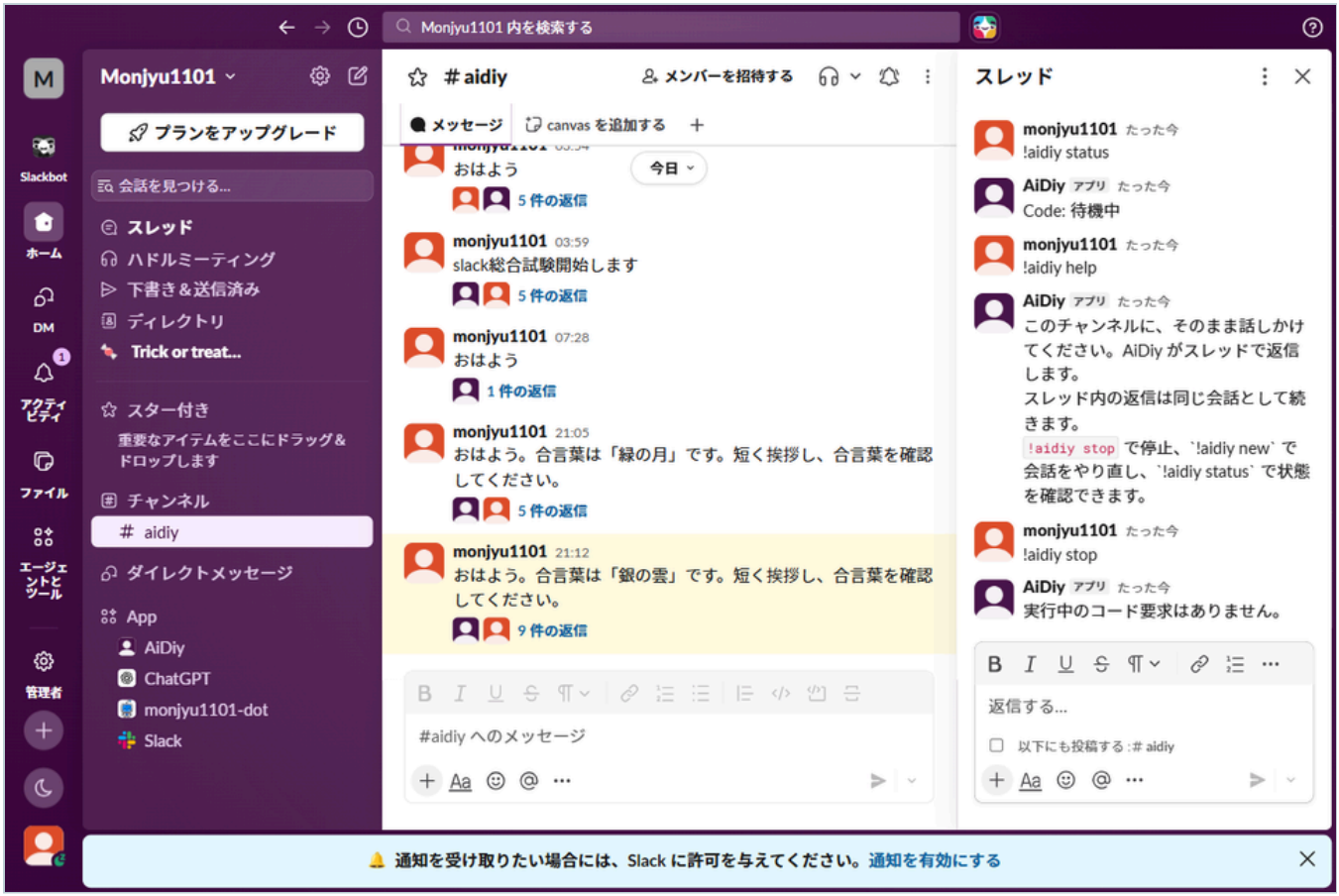
!aidiy helpを送り、専用チャンネルへの話しかけ方が返りました。

図502：ヘルプ表示



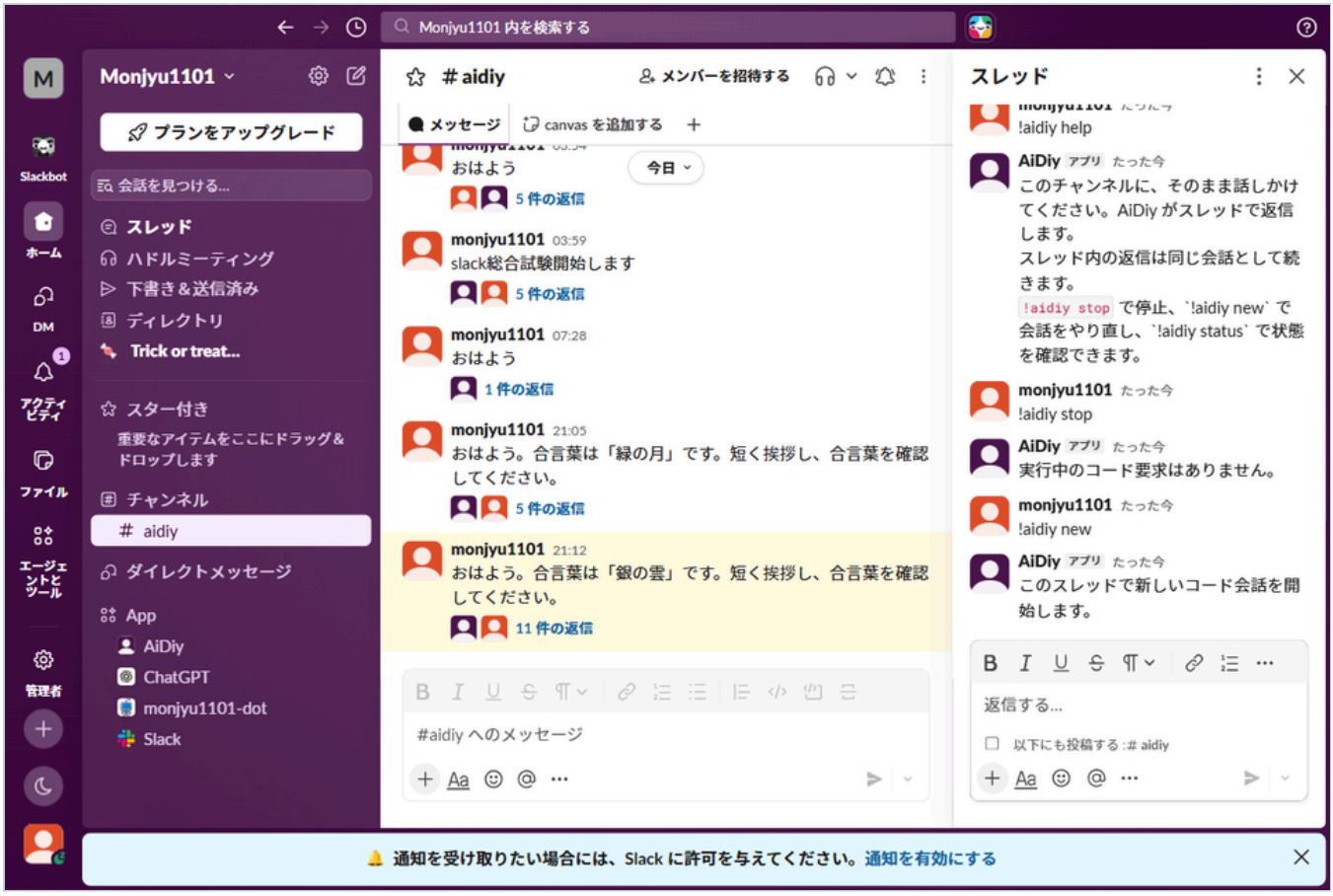
処理がない状態の`!aidiy stop`には「実行中のコード要求はありません。」が返りました。実行中の要求を中断した確認とは区別します。

図503：待機中の停止



!aidiy newに、新しいコード会話を開始する案内が返りました。

図504：会話のリセット



単一起動

接続中にaidiy_slackをもう一度起動しました。同じパネルが表示され、ウィンドウIDとパネル1枚の維持を照合しました。

図505：単一起動



確認した条件

ケース	確認済みの結果
SL-101 Diagミニ表示から起動	Slackアイコンからパネルが開く
SL-102 自動接続	保存済み設定で接続し、接続済み・CODE ON・モデルを表示
SL-103 二重起動の防止	パネル・接続が増えず、既存パネルを表示
SL-201 投稿への応答	「おはよう」へのAI回答をパネルとSlackで確認
SL-202 スレッドでの継続	Hermesの既定モデルで前の投稿を踏まえた回答
SL-203 状態確認	同じスレッドのstatusに「Code: 待機中」と回答
SL-301 接続中のモデル設定	接続を続けたまま設定欄と適用・取消を表示
SL-302 適用して再接続	選択の保存と再接続を確認。現行の固定コード欄とautoの保存値を確認

ケース	確認済みの結果
SL-303 切断と再接続	未接続の設定画面へ戻り、再び接続済みになる
SL-304 終了	パネル・Bot・子プロセスを回収
SL-305 接続中の終了確認	確認のキャンセルは接続継続、終了は全プロセスを回収。切断中は確認なしで終了
SL-401 初回起動	保存値がない条件では設定画面で停止し、接続前に保存ファイルを作らない
SL-402 保存と起動	「接続」でaidiy_hermesとautoを保存し、終了後の起動で前回値の自動接続

通信方式と表示

通常モード：3秒間隔。 起動直後と会話直後は3秒ごとにポーリングします。**最後の会話から10分でアイドリングモード：6秒間隔。** 新しい会話を受信すると3秒へ戻ります。許可された利用者の投稿とBotの回答を会話として扱い、回答待ちの間も通常間隔を保ちます。通信失敗時は別の再試行待機を使います。

アイドリングの6秒間隔は、利用者指定により実時間待機の試験を省略しています。時計を進める隔離試験で境界と復帰を確認しました。アイコンの上下動は12秒周期、周囲の四角形は40秒で1回転し、動きを半分の速度にしています。

自動・隔離試験と限界

Nodeの82件、Pythonの14件、DiagのSlack起動7件が成功しました。型検査と配布画面の一致も確認しています。自動試験は合成通信や一時設定を含み、実サービスの確認は各章の投稿・AI回答・画面結果で判定します。

codex_cli/…では、スレッドの続きで前の投稿を参照しなかった参考結果があります。会話継続の合格はHermesのautoでの確認です。Slack版には音声会話機能がありません。

VPN・社内プロキシ経由の実接続、Mac／Linux実機、長時間連続運転は未実施・判定対象外です。実行中の長いコード要求の中断は、実チャンネルでは未実施です。

終了時の状態

モデル設定は元の内容へ復元し、試験用パネルとBotを終了しました。Diagと共有サービスは継続稼働しています。試験投稿は専用チャンネルに残しています。ブラウザの元の下書きも復元しました。

長押しドラッグの実機確認（2026-11-02）

Windows実機で mouse_long_press_drag（保持0.8秒、移動0.6秒。Browser全画面は移動0.8秒）を使用。直前の画面から対象を同定し、移動後の実画面とOSの窓位置を照合します。入力欄・ボタンの除外判定は下記の実施対象に限定します。位置は画面座標の (x,y) です。

既存ランチャーの --no-connect で起動し、未接続の標準窓 (650,10) → (710,50) の背景移動を実機確認。ピンボタン上の長押しからボタン外へ移動・解放しても窓が移動せず、ピン状態も変わらない。Bot接続・投稿は未実施。

解放後にポインターを離して追従が残らないこと、通常UIが使えることを確認。Diag以外の今回起動した窓は通常UIで終了しました。1〜2pxの寸法丸めは補正を追加していません。全セルフテストの再実行や、未実施モードの合格を意味しません。

未接続での追加実機確認（2026-11-02）

ルートcwdの正式 `aidiy_slack --no-connect --project "C:¥Users¥admin¥Desktop¥AiDiy2026_dev"` で起動。未接続・対象プロジェクト、コードaidiy_hermes固定、モデルautoを確認しました。モデル一覧を開き、候補を目視後Escで閉じ、autoと未接続を維持。設定変更・保存・接続・投稿なしで×終了しました。Nanami1.2の事前音声を直列再生。第1章の未接続起動、第3章の候補表示と無変更復帰、第4章の未接続終了に限定したPASSです。接続中の取消／再接続、スレッド投稿・応答・実サービスコマンドは今回未実施で、宛先と接続範囲の確認が必要です。

証拠照合と残る確認

2026-11-03の文書・既存証拠照合です。新しい実機試験ではありません。

- **確認済み**：SL-101〜103、201〜203、301〜305、401〜402と各図で、実Slackの接続・挨拶・同スレッド継続・状態確認・保存/再接続・取消/終了は確認済みです。元日時は本書で未記載。2026-11-02の--no-connect確認だけを採用して過去投稿を未実施へ戻しません。
- **追加実機確認前の記録（現在の判定は未尾）**：長いコード要求の実チャンネル中断は明記された未実施です。試す場合はworkspace/channel/thread・有限依頼・対象だけの中止と処理終了を先に固定します。アイドリング実時間試験は利用者指定で省略し時計模擬で確認済みなので、必須残件へ戻しません。全サービス再投稿は不要です。
- **中断実機確認前の読取履歴（現行残件ではない）**：第5章の図503は待機中の `!aidiy stop` に「実行中のコード要求はありません。」が返った証拠であり、実行中の要求を中断したPASSではありません。本文の「実チャンネルでは未実施」を維持します。既存結果JSON・台帳にも実行中中断の成功証拠は確認できませんでした。設定済みのworkspace（team）とchannel識別子は取得可能ですが、今回の試験投稿先・有限の依頼内容を具体的に指定した承認記録は確認できません。設定済みであることを投稿承認とは扱わず、**投稿先/内容未指定のため未実施（保留）**とします。 対象threadも未指定です。 合格条件は、指定した専用先で有限要求1件の実行中を確認し、その要求だけを中断して処理終了を照合することです。今回は読取りのみで、投稿・要求開始・中断操作・AI送信はいずれも0回、設定・ソース変更なし。通常会話の過去成功を残件へ戻していません。

実機確認前の隔離テスト記録

既存のcode.test.tsとbot.test.tsから中断関連だけを選択し、6件PASS・0失敗・0skip。未開始要求の取消、仮返信待機中の停止、同一スレッドの実行・待ち行列の停止と別スレッド保持、停止後の新要求、繰返しstop、偽CLI回収を確認しました。外部投稿・AI送信0回。実Slackチャンネルでの中断は未確認であり、投稿先・有限依頼内容の指定が必要です。 証拠JSON・TAPログ。

実チャンネルの実行中要求中断（2026-11-03）

ユーザーが専用チャンネルの挨拶スレッド内へ全角「！stop」を投稿。thread 1793691311.958579はUTC 07:35:31.623停止→07:35:33.068停止受理、仮返信を07:35:33に停止完了へ更新。thread 1793691392.301439は07:36:43.286停止→07:36:44.723停止受理→07:36:45に停止完了へ更新。公式履歴APIの本文・同じthread_ts・edited.tsと、runnerのchild close後の結果をchat.updateする実装を照合し、2回とも実行中要求中断PASSです。返信のtsは仮返信作成時刻なので、中止より前に完了したと誤判定しません。別の新規投稿として送ったthread 1793691319.301959のstopは対象なしであり、この1回は中断PASSに含めません。

前節の「投稿先/内容未指定による未実施」はこの実施前の履歴です。今回のユーザー自身の投稿による確認で、**実行中要求中断を残件から外します**。終了後に対象Hermes/oneshotプロセスの残留なしを確認しました。こちらから投稿・再実行は0回、ソース・設定変更なし。起動ログは空のため成功根拠にせず、実サービスのメッセージ履歴と処理実装を採用。音声/他OS/全モデルの未確認範囲は変更しません。証拠JSON。

現行の判定（文書順再照合）

- **確認済み**：通常会話とユーザー投稿による実要求中断2回、隔離中断6PASS。
- **本当に残る確認・範囲**：追加必須残件なし。通常会話と待機中stopだけを中断証拠にしていない。
- **証拠の扱い**：上記の本文・図・参照結果を照合。先行調査の未実施/拒否/準備失敗は当時の記録であり、後続の成功や実施不要指定を優先します。今回の文書整理を新しい実機PASSには数えません。